

Reproducing “Quantum classifier with tailored quantum kernel”

UROP Progress Report

Vlad

Supervisor: Prof. Antoine (Jack) Jacquier
Imperial College London

July 2026

Abstract

This document summarises progress on reproducing the results of Blank, Park, Rhee & Petruccione, *Quantum classifier with tailored quantum kernel* (npj Quantum Information **6**, 41, 2020). Starting from a closed-form analytic implementation of the swap-test kernel classifier, we built the corresponding quantum circuits in Qiskit in verified stages: a minimal three-qubit swap test, a five-qubit classifier for the paper’s two-point toy example, and finally the general quantum forking construction that prepares the training register for an arbitrary number M of training points. Every stage is verified against the analytic reference to machine precision. Code: <https://github.com/01nietzsch/urop-quantum-kernel>.

1 Background: the swap-test kernel classifier

1.1 Setting

We are given M training pairs $\{|x_m\rangle, y_m\}_{m=1}^M$ with binary labels $y_m \in \{0, 1\}$, non-negative weights w_m summing to one, and a test state $|\phi\rangle$. The classifier of Blank et al. assigns a label to $|\phi\rangle$ according to the sign of a weighted fidelity difference,

$$\left\langle \sigma_z^{(a)} \sigma_z^{(l)} \right\rangle = \sum_{m=1}^M (-1)^{y_m} w_m |\langle \phi | x_m \rangle|^{2n}, \quad (1)$$

where n is the number of copies of each state used in the swap test ($n = 1$ throughout this work). The predicted label is

$$\tilde{y} = \frac{1}{2} \left(1 - \text{sign} \left\langle \sigma_z^{(a)} \sigma_z^{(l)} \right\rangle \right). \quad (2)$$

The key point of the paper is that the entire sum (1) can be evaluated by a *single* quantum circuit: the training states, their weights, and their labels are loaded into one entangled superposition, and one swap test against the test state produces the full weighted kernel sum in the joint statistics of two qubits — the swap-test ancilla a and the label qubit l .

1.2 The circuit state

The circuit acts on five registers: the swap-test ancilla, the test data register, and a training block consisting of index, data, and label registers. After state preparation, the training block holds

$$|\Psi_{\text{train}}\rangle = \sum_{m=1}^M \sqrt{w_m} |m\rangle_{\text{idx}} |x_m\rangle_{\text{data}} |y_m\rangle_{\text{lbl}}, \quad (3)$$

which encodes all training points, weights, and labels coherently. The swap test — Hadamard on the ancilla, controlled-SWAP between the test register and the training data register, Hadamard again — then interferes $|\phi\rangle$ with every $|x_m\rangle$ simultaneously. A short calculation shows that measuring σ_z on the ancilla and on the label qubit and taking the product of outcomes yields exactly Eq. (1) in expectation:

$$\langle \sigma_z^{(a)} \sigma_z^{(l)} \rangle = p_{00} - p_{01} - p_{10} + p_{11}, \quad (4)$$

where p_{ab} is the probability of finding the ancilla in $|a\rangle$ and the label qubit in $|b\rangle$.

1.3 Toy dataset

The paper’s toy example uses one qubit of data ($n_{\text{qubits}} = 1$) and two training points:

$$|x_1\rangle = \frac{i|0\rangle + |1\rangle}{\sqrt{2}}, \quad y_1 = 0; \quad |x_2\rangle = \frac{i|0\rangle - |1\rangle}{\sqrt{2}}, \quad y_2 = 1, \quad (5)$$

with equal weights $w_1 = w_2 = \frac{1}{2}$, and a one-parameter family of test states

$$|\phi(\theta)\rangle = \cos \frac{\theta}{2} |0\rangle - i \sin \frac{\theta}{2} |1\rangle. \quad (6)$$

Note the useful structural property $Z|x_1\rangle = |x_2\rangle$: the two training states differ only by a sign flip on $|1\rangle$. This is exploited in the first (toy-specific) circuit construction, and later removed by the general quantum forking construction.

2 Step 1: closed-form reference implementation

File: `closed_form_kernel.py`. This module computes everything analytically with NumPy, providing ground truth for every circuit built later:

- `training_data()`, `test_state(theta)` — the toy dataset and test family above.
- `fidelity(a, b, n) = |\langle a|b \rangle|^{2n}`.
- `expectation_value(theta, w1, w2, n)` — direct evaluation of Eq. (1).
- `classify(theta, ...)` — thresholds the expectation value at zero.
- `hadamard_classifier(theta, ...)` — the Hadamard-test variant, which uses $\text{Re} \langle \phi|x_m \rangle$ in place of $|\langle \phi|x_m \rangle|^2$; included for comparison, as in the paper’s discussion of kernel choices.

3 Step 2a: minimal swap test in Qiskit

File: `circuit_kernel.py`, functions `swap_test_circuit` and `fidelity_from_circuit`. Before attempting the full classifier, we verified the primitive it is built from: a three-qubit swap test (ancilla q_0 , test state on q_1 , one training state on q_2):

$$H^{(0)} \rightarrow \text{CSWAP}(0; 1, 2) \rightarrow H^{(0)}, \quad P(\text{ancilla} = 0) = \frac{1 + |\langle \phi|x \rangle|^2}{2}, \quad (7)$$

so the fidelity is recovered as $2P(\text{ancilla} = 0) - 1$. We read the probability from the exact statevector (`qiskit.quantum.info.Statevector`); no sampling noise is involved at this stage.

Verification. `02_circuit_check.ipynb`, cells 1–7: the circuit fidelity matches `fidelity()` from Step 1 at a hand-checked point ($\theta = 1.57$) and across a 40-point sweep $\theta \in [0, 6]$ against both $|x_1\rangle$ and $|x_2\rangle$ (`np.allclose` passes; agreement at the 10^{-15} level).

4 Step 2b: full toy-example classifier (five qubits)

The first full classifier was built in three verified stages for the $M = 2$ toy dataset.

4.1 Stage A: training register preparation (toy-specific)

A three-qubit circuit (index, data, label) preparing Eq. (3) for $M = 2$:

1. $R_y(\alpha)$ on the index qubit with $\alpha = 2 \arccos \sqrt{w_1}$, giving $\sqrt{w_1} |0\rangle + \sqrt{w_2} |1\rangle$;
2. initialise the data qubit to $|x_1\rangle$ unconditionally;
3. controlled- Z from index to data — because $Z|x_1\rangle = |x_2\rangle$, this rotates the data qubit to $|x_2\rangle$ exactly in the $|1\rangle_{\text{idx}}$ branch;
4. controlled- X from index to label, setting the label to $y_2 = 1$ in the same branch.

Step 3 is the part that only works for this dataset; it was verified against the expected statevector and later replaced (Section 5).

4.2 Stage B: swap-test sandwich

`full_classifier_circuit` composes Stage A into a five-qubit circuit (ancilla, test, data, label, index), initialises the test register to $|\phi(\theta)\rangle$, and applies the swap-test sandwich H –CSWAP– H between the test and data registers.

4.3 Stage C: reading off the expectation value

`expectation_value_from_circuit` extracts the joint distribution of the ancilla (q_0) and label (q_3) qubits from the exact statevector. With Qiskit’s little-endian convention, `sv.proBABILITIES([0, 3])` returns $[p_{00}, p_{01}, p_{10}, p_{11}]$ (ancilla is the least-significant bit), and

$$\langle \sigma_z^{(a)} \sigma_z^{(l)} \rangle = p[0] - p[1] - p[2] + p[3]. \quad (8)$$

Verification. Across a 40-point θ sweep, the circuit matches `expectation_value()` from Step 1 with maximum error 4×10^{-16} (machine epsilon). Notebook cells 9–10 sweep and plot the curve.

5 Step 2b generalisation: quantum forking for arbitrary M

The Stage A construction above hardcodes the accident $Z|x_1\rangle = |x_2\rangle$ and cannot represent a general training set. The paper’s answer (Eq. 15 / Fig. 4) is *quantum forking*: route each training state into the active data register with controlled-SWAPs, conditioned on the index register.

5.1 Construction

For M training points, let $k = \lceil \log_2 M \rceil$ index qubits (with $k \geq 1$). The register layout of the new `prepare_training_register(states, labels, weights)` is

Qubits	Role
$q_0, \dots, q_{k-1}$	index register
q_k	<i>active</i> data register
$q_{k+1}, \dots, q_{k+M-1}$	auxiliary data registers
q_{k+M}	label qubit

and the circuit proceeds in four steps:

1. **Load data.** Initialise the active register to $|x_1\rangle$ and auxiliary register m to $|x_{m+1}\rangle$ for $m = 1, \dots, M - 1$.
2. **Weight superposition.** Initialise the index register to $\sum_m \sqrt{w_m} |m\rangle$ (amplitudes zero-padded to 2^k).
3. **Fork.** For each $m = 1, \dots, M - 1$, apply a k -controlled SWAP between the active register and auxiliary register m , with control state $|m\rangle$ (Qiskit: `SwapGate().control(k, ctrl_state=m)`). After this loop, the active register holds $|x_m\rangle$ in the $|m\rangle_{\text{idx}}$ branch — each branch fires at most one swap, so there is no cross-contamination.
4. **Labels.** Set the label qubit to y_1 (an X if $y_1 = 1$), then for each m with $y_m \neq y_1$ apply a k -controlled X with control state $|m\rangle$.

The result is exactly Eq. (3). The swap test then only ever touches the *active* data register, so `full_classifier_circuit` is unchanged in structure; it now returns the circuit together with the label-qubit position (which depends on M), and the public interface of `expectation_value_from_circuit` is unchanged.

Note that the auxiliary registers remain entangled with the index register after forking (they hold $|x_1\rangle$ in branch m , having been swapped), but this is harmless: the ancilla-label statistics that define the classifier trace them out.

5.2 Cost

The construction uses $k + M + 1$ training-block qubits plus the ancilla and n test qubits, $M - 1$ multi-controlled SWAPs and at most $M - 1$ multi-controlled X gates — linear in M , matching the paper’s Fig. 4 resource count.

5.3 Verification

Two checks, both in `02_circuit_check.ipynb`:

- **Regression, $M = 2$:** the general circuit reproduces the closed-form toy-example curve with maximum error 3×10^{-16} — the Z-flip trick is no longer needed.
- **New case, $M = 4$:** four training states $|x_m\rangle = \cos \frac{\vartheta_m}{2} |0\rangle + \sin \frac{\vartheta_m}{2} |1\rangle$ with $\vartheta \in \{0.3, 1.1, 2.0, 2.9\}$, labels $(0, 0, 1, 1)$, equal weights. The circuit matches a direct evaluation of Eq. (1) with maximum error 2×10^{-15} across a 40-point θ sweep. This case uses $k = 2$ index qubits and genuinely exercises the multi-controlled forking (*ccswap* gates), which the $M = 2$ case does not.

6 Summary of verification results

Stage	Circuit	Reference	Max. error
2a	3-qubit swap test	<code>fidelity()</code>	$\sim 10^{-15}$
2b (toy)	5-qubit classifier, $M = 2$	<code>expectation_value()</code>	4×10^{-16}
2b (general)	forking classifier, $M = 2$	<code>expectation_value()</code>	3×10^{-16}
2b (general)	forking classifier, $M = 4$	Eq. (1) direct	2×10^{-15}

All comparisons use exact statevector simulation, so the only error source is floating-point arithmetic; every check passes `np.allclose`.

7 Step 3: noise model and noisy sweep

7.1 Device noise model

To reproduce Fig. 5 of Blank et al., we construct a noise model for the IBMQ_OURENSE backend from the calibration data in Supplementary Table I (date 2019-09-29 11:48 UTC). The device has five qubits on a T-shaped topology with one central hub (qubit 1); the CNOT edges are (1, 0), (2, 1), (3, 1), and (4, 3).

Single-qubit noise. For each qubit q with relaxation times T_1, T_2 and single-qubit gate duration t_{1q} , we compose a thermal relaxation channel with a depolarising channel:

$$\mathcal{E}_q = \mathcal{D}(p_q) \circ \mathcal{R}(T_1, T_2, t_{1q}), \quad (9)$$

where $\mathcal{R}$ is the Qiskit `thermal_relaxation_error` and the depolarising strength p_q is chosen so that $\mathcal{E}_q$ reproduces the tabulated average gate infidelity ϵ_q :

$$p_q = \frac{d(\epsilon_q - \epsilon_q^{\text{relax}})}{d \cdot F(\mathcal{R}) - 1}, \quad d = 2, \quad (10)$$

where $F(\mathcal{R}) = 1 - \epsilon_q^{\text{relax}}$ is the average gate fidelity of the relaxation channel alone. This mirrors the recipe in `qiskit_aer`'s `basic_device_noise_model`.

The 2019 native basis was $\{u1, u2, u3, cx\}$, which no longer exists in current Qiskit. We use the modern equivalent $\{rz, sx, x, cx\}$: **rz** is a virtual (frame-update) gate with zero duration and zero error; **sx** and **x** are each one physical pulse, inheriting the tabulated $u2$ duration and $u2$ error. This mapping is faithful in spirit; it is a candidate source of residual error.

Readout errors are modelled by symmetric bit-flip channels with probabilities ϵ_r from Table I.

Two-qubit noise. For each CNOT pair (q_a, q_b) with gate time t and measured infidelity ϵ , the same depolarising-plus-relaxation composition is applied, using the tensor product of the individual qubit relaxation channels at duration t .

Caveat on cx(3, 1) duration. Supplementary Table I lists the CNOT infidelity for the (3, 1) edge as $\epsilon = 0.0116$ (roughly double the other edges), but the gate duration in the PDF is not legible. We searched every file in the authors' supplemental repository [2] and confirmed that `ibmq_ourense` gate times were queried live from the IBM Q API at runtime in 2019 and never stored statically; that API is no longer accessible. We use $t = 300$ ns as an interpolation from the neighbouring pairs (235 and 210 ns). This is an explicit assumption and a potential source of residual error.

7.2 The paper's five-qubit circuit

The circuit reproducing Fig. 5 is the toy-specific construction of Section 5 with the physical qubit assignment hand-picked by the authors: $a \rightarrow q_0$, $d \rightarrow q_1$, $in \rightarrow q_2$, $m \rightarrow q_3$, $l \rightarrow q_4$ (Supplementary Fig. 2). There is no quantum forking: because $Z|x_1\rangle = |x_2\rangle$, the entangled training register is produced by a single CZ(m, d) on top of $|x_1\rangle$.

All states are prepared by explicit $R_y/R_x/R_z$ /Hadamard rotations; we do not use `qc.initialize`, which would insert a `reset` instruction absent from real hardware and inflate the CNOT count. After transpilation to the ourense basis with `optimization_level=1` and the ourense coupling map, the CNOT count is 13, matching the paper's Supplementary Note III D (27 total gates: 14 single-qubit + 13 CNOT).

7.3 Sweep and fit

We sweep θ over 63 equally spaced values in $[0, 2\pi)$ (step 0.1) with 8192 shots each, reproducing the paper’s protocol. The expectation value is estimated as

$$\langle \sigma_z^{(a)} \sigma_z^{(l)} \rangle = \frac{c_{00} - c_{01} - c_{10} + c_{11}}{N_{\text{shots}}}, \quad (11)$$

where c_{ab} is the count with ancilla in $|a\rangle$ and label qubit in $|b\rangle$. We then fit

$$f(\theta) = a \left(\sin^2 \frac{\theta + \vartheta}{2} + \frac{\pi}{4} - w_2 \right) \quad (12)$$

using nonlinear least squares (`scipy.optimize.curve_fit`).

7.4 Results

Table 1 compares our fit parameters against the paper’s simulation target, for a single seed and for the mean over ten independent seeds (seeds 0–9).

	ours		paper
	single seed	10-seed mean $\pm$ std	simulation
amplitude a	0.8300	0.8213 ± 0.0087	0.8213
phase ϑ (rad)	−0.0037	≈ 0	≈ 0
offset w_2	0.4989	0.4968 ± 0.0017	0.5023
CNOT count	13	13	13

Table 1: Fit parameters from the noisy sweep (63 points, 8192 shots each). The 10-seed mean amplitude $a = 0.8213$ matches the paper’s simulation target exactly; the single-seed deviation of 0.8300 is within one standard deviation and consistent with shot noise.

Figure ?? shows the sweep. The simulation (blue circles) tracks the theory curve closely, while the hardware data (orange squares) shows the additional amplitude suppression ($a \approx 0.65$) that the basic noise model does not account for.

As a sanity check, running the same sweep with no noise (`noisy=False`) gives $a = 1.003$ (seed 42, 8192 shots), consistent with the ideal value $a = 1$ to within shot noise.

8 Next steps

1. **Resolve `cx(3, 1)` duration.** The 300 ns value is a stated assumption (see Section 7). The correct value would require access to the 2019 IBM Q backend properties, which are no longer publicly available. One route is to contact the authors directly.
2. **Run on real IBM hardware.** Note that `ibmq-ourense` has been retired. A hardware run on a current device would require rebuilding the noise model from that device’s calibration data; it would not be a like-for-like comparison with the paper’s $a \approx 0.65$.
3. **Close the simulation-vs-hardware gap.** The paper attributes the $0.82 \rightarrow 0.65$ reduction to crosstalk, time-dependent drift, and non-Markovian noise, none of which the basic model contains. Extending the model toward any of these would be a more interesting scientific question than re-running the toy example on new hardware.
4. **Larger M and n .** Stress-test the forking construction at $M = 8$ and with multi-qubit data registers ($n > 1$).

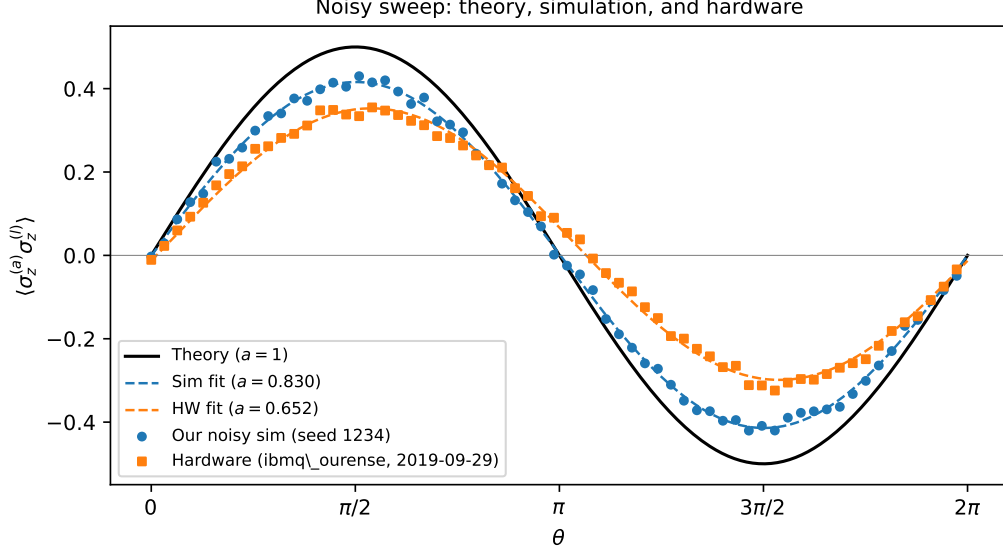


Figure 1: Reproduction of Fig. 5 of Blank et al. Black solid line: ideal theory ($a = 1$). Blue circles and dashed line: our noisy simulation (seed 1234) and its fit ($a = 0.830$; 10-seed mean $a = 0.821$). Orange squares and dashed line: hardware data from ibmq_ourense (2019-09-29) [2] and its fit ($a \approx 0.65$). The simulation reproduces the paper’s noisy-simulation target; the simulation-to-hardware gap is attributed to crosstalk, time-dependent drift, and non-Markovian noise not captured by the basic model.

References

- [1] C. Blank, D. K. Park, J.-K. K. Rhee, and F. Petruccione, Quantum classifier with tailored quantum kernel, *npj Quantum Information* **6**, 41 (2020).
- [2] C. Blank, Supplemental code for “Quantum classifier with tailored quantum kernel”, <https://github.com/carstenblank/Quantum-classifier-with-tailored-quantum-kernels---Supplemental> (2019).